

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Конструювання програмного забезпечення»

Робота №1

Тема «Знайомство з Java»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Леощенко С.Д.

2025

МЕТА

Вивчити основні можливості та принципи роботи з мовою Java.

ЗАВДАННЯ

1. Ознайомитися з основними теоретичними відомостями за темою роботи, використовуючи ці методичні вказівки, а також рекомендовану літературу.

2. Вивчити основні принципи роботи з Java.

3. Виконати наступні завдання:

Загальні завдання:

1. Ввести з консолі n цілих чисел і помістити їх у масив. На консоль вивести числа-паліндроми, значення яких у прямому й зворотному порядку збігаються.

2. Шляхом перестановки елементів квадратної матриці дійсних чисел домогтися того, щоб її максимальний елемент перебував у лівому верхньому куті, наступний по величині – у позиції (2,2), наступний по величині – у позиції (3,3) і т.д., заповнивши в такий спосіб всю головну діагональ.

Індивідуальні завдання:

1. Упорядкувати рядки (стовпці) матриці в порядку зростання значень елементів k -го стовпця (рядка).

2. Знайти й вивести найбільше число зростаючих (спадаючих) елементів матриці, що йдуть підряд.

3. Транспонувати квадратну матрицю.
4. Повернути матрицю на 90 (180, 270) градусів проти часової стрілки.
5. Побудувати матрицю, віднімаючи з елементів кожного рядка матриці її середнє арифметичне.
6. Ущільнити матрицю, видаляючи з неї рядки й стовпці, заповнені нулями.
7. Перетворити рядки матриці таким чином, щоб елементи, які дорівнюють нулю, розташовувалися після всіх інших.
8. Знайти кількість всіх седлових точок матриці. (Матриця A має седлову точку $A_{i,j}$, якщо $A_{i,j}$ є мінімальним елементом в i -ому рядку й максимальним в j -ому стовпці).
9. Знайти число локальних мінімумів. (Сусідами елемента матриці назвемо елементи, що мають із ним загальну сторону або кут. Елемент матриці називається локальним мінімумом, якщо він строго менше всіх своїх сусідів).
10. Перебудувати задану матрицю, переставляючи в ній стовпці так, щоб значення їхніх характеристик убували. (Характеристикою стовпця прямокутної матриці називається сума модулів його елементів).

4. Оформити звіт з роботи.
5. Відповісти на контрольні питання.

ТЕОРІЯ

На відміну від Python у Java помітно кілька відмінностей:

- Весь код має бути у класах (як у C#);

- Для виведення тексту у консоль інша функція: `System.out.println` (замість `print`);
- Всі змінні мають бути типізованими.

Після спроби запуску програми з'ясовано: назва головного класу (той, в якому мутод `main`) має співпадати з назвою файлу.

Блоки коду у Java визначаються фігурними дужками `{ }` (як у C++).

У головному класі потрібен метод `main` з модифікатором `static`:

```
public static void main(String[] args)
```

При спробі оголосити змінну з плаваючою крапкою:

```
float pi = 3.14;
```

редактор видає помилку “cannot convert from double to float”.

Працює після додавання постфіксу `f` до числа:

```
float pi = 3.14f;
System.out.println(pi);
> 3.14
```

Числові константи за замовчуванням зберігаються як `double`.

У мові Java не можна записувати рядки у одинарних лапках `' '`. Ці символи (як у C++) використовуються лише для типу `char`.

Для рядків обов'язкове використання подвійних лапок `" "`.

Тип даних `string` потрібно оголошувати з великої літери.

Конкатенація рядків (як в Python) працює:

```
String plus = "One" + "two";
System.out.println(plus);
> Onetwo
```

У назві методу (чи змінної) не можна ставити цифру на перше місце.

Як назви можна використовувати Українські літери:

```
int привіт = 3;  
System.out.println(привіт);  
> 3
```

У мові Java не можна використовувати рядки (або числа) для порівняння з типом `boolean`:

```
boolean interesting = 0 == false;
```

Оператори порівняння працюють як у C++:

```
boolean one = false;  
boolean two = true;  
boolean res = one || two;  
System.out.println(res);  
> true
```

Кількості байтів для типів даних:

- byte: 1;
- short: 2;
- int: 4;
- long: 8;
- char: 2.

Оператори схожі на C++.

У мові Java кожен рядок потрібно завершувати символом [середнику](#).

Також можна не додавати фігурні дужки якщо операція займе лише один рядок:

```
int one = 2;
if (one != 2)
    System.out.println("None");
else
    System.out.println("True");
> True
```

При спробі створити словник (як в Python) виявлено: примітивні типи не можна використовувати у колекціях.

Для ітерації через геш-таблицю Java до об'єкта можна застосувати метод `forEach`:

```
Map<Integer, Integer> testCases = Map.of(1, 1, 2, 4, 7, 3);
testCases.forEach((k, v) -> {
    System.out.println(k + " and " + v);
});
> 1 and 1
> 2 and 4
> 7 and 3
```

Якщо метод розташований нижче у класі, його можна використати у методах, які знаходяться вище по класу:

```
public static void main(String[] args) {
    ...
    testCases.forEach((k, v) -> {
        int res = testTarget(k, v);
    ...
}

public static int testTarget(int value, int target) {
    int result = 0;
    ...
}
```

В мові Java мітки можна використовувати, наприклад, для завершення зовнішнього циклу у внутрішньому:

```
outer: for (int i = 0; i < 3; i++) {
    inner: for (int j = 1; j <= 3; j++) {
        if (i == 2)
            continue outer;
        else
            System.out.println(i + " and " + j);
    }
}
```

```
> 0 and 1
> 0 and 2
> 0 and 3
> 1 and 1
> 1 and 2
> 1 and 3
```

Так само з інструкцією `break`:

```
boolean check = false;
outer: while (!check) {
    System.out.println("False");
    boolean other = false;
    inner: while (!other) {
        System.out.println("False" + " False");
        // Множення рядків не працює. Якщо писати "False"*2, видасть
        // помилку
        other = true;
        if (other) {
            break outer;
        }
    }
}
> False
> False False
```

Інструкція `break` перериває цикл, а `continue` пропускає ітерацію.

Конструкція `switch` працює, схоже, як у C++:

```
for (int i = 0; i <= 3; i++) {
    switch (i) {
        case 0:
            System.out.println("One");
            break;
        case 1:
            System.out.println("Two");
            break;
        case 2:
            System.out.println("Three");
            break;

        default:
            break;
    }
}
```

У циклі `for` ітерація (зміна ітератора) відбувається перед кінцем ітерації.

```
// Якщо написати i<=2, j<=3 працювати не буде
for (int i = 0, j = 1; i <= 2 && j <= 3; i++, j++) {
    System.out.println(i + " and " + j);
}
```

Примітивні типи даних є вбудованими в мову Java, а непримітивні (посилальні) є переважно створеними користувачами.

Приклади примітивних типів: int, char, boolean.

Приклади посилальних типів: String, Array, Map.

Джерело: [Java Non-Primitive Data Types at W3Schools.com](https://www.w3schools.com/java/java_non_primitive_data_types.asp)

Масиви мають бути:

- 1) оголошені: прописані типом змінних та назвою;
- 2) визначені: створені ключовим словом `new`;
- 3) ініціалізовані: заповнені даними.

Приклад:

```
// Оголошення
String[] texts;
// Встановлення
texts = new String[3];
// Ініціалізація
texts[0] = "One";
texts[1] = "Two";
texts[texts.length - 1] = "Three";
// Вивід
for (String text : texts) {
    System.out.println(text);
}
> One
> Two
> Three
```

Двомірні масиви в мові Java задаються схоже на одномірні:

```
int[][] grid = new int[3][3];
int counter = 0;
for (int i = 0; i < grid.length; i++) {
    for (int j = 0; j < grid.length; j++) {
        grid[i][j] = counter;
    }
}
```



```
        counter += 1;
    }
}

System.out.println("Grid:");
for (int i = 0; i < grid.length; i++) {
    for (int j = 0; j < grid.length; j++) {
        System.out.print(grid[i][j] + " ");
    }
    System.out.println();
}
> Grid:
> 0 1 2
> 3 4 5
> 6 7 8
```

Для використання мови Java на Windows необхідно:

- 1) Java Runtime Environment (JRE) для запуску;
- 2) Java Development Kit (JDK) для розробки.

JDK включає в себе Java Virtual Machine – програму для запуску байткоду (файлів з типом *.class*). Компілятор Java спочатку створює байткод, а тоді JVM перекладає його на рідний код для платформи, на якій вона встановлена.

Через це програміст може написати код один раз і запускати його на всіх платформах, які підтримують JVM, очікуючи таких самих результатів виконання.

ВИКОНАННЯ

0 Підготовка до роботи

Для використання Java на комп'ютері було встановлено JDK (Java Development Kit – [Download Java at Oracle](#)). JDK є пакетом застосунків для запуску та роботі на мові Java. Він зазвичай включає JRE (Java Runtime Environment) та JVM (Java Virtual Machine), а також компілятор *javac*, архіватор *jar* та інше.

Після завантаження та встановлення JDK 23 версії x64 від Oracle було додано теку мови Java до системних змінних середовища (*environmental variables*):

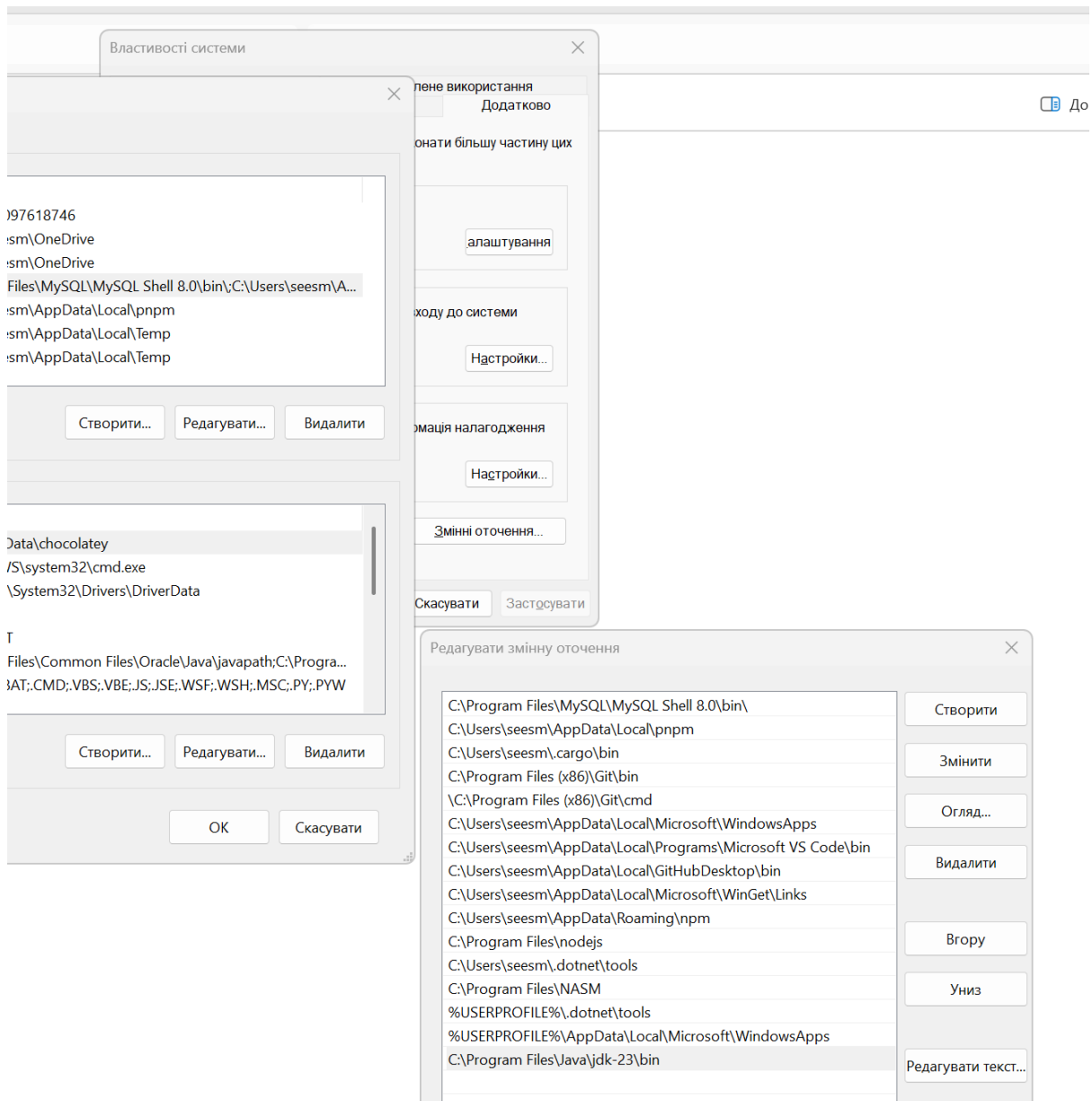


Рисунок 0.1 – Додана тека Java до змінних середовища

Це потрібно для того, аби кожного разу при запуску програми не писати шлях до цієї теки, а лише використовувати ключове слово *javac*.

Далі для роботи з Java у середовищі Visual Studio Code було встановлено спершу (здається) розширення “Java” від Oracle Corporation.



Java

Oracle Corporation  [oracle.com](https://www.oracle.com) |  2,398

Java Platform Extension for Visual Studio Code

Install



Auto Update



DETAILS

FEATURES

CHANGELOG

Java Platform Extension for Visual Studio Code

Java Platform extension from Oracle brings full featured development support (edit-compile-debug & test cycle) to VS Code. It also offers support for Maven and Gradle projects. Applications using JDK 8 and above are supported.

Getting Started

1. Setting up the JDK

- If no JDK is present in your system then the extension can set things up for you. For more details refer to [JDK Downloader](#) section.
- Set the JDK in the **View | Command Palette | Preferences: Open User Settings | Jdk: Jdkhome** setting to point to

Marketplace

Identifier	oracle.oracle-java
Version	23.1.0
Published	2023-10-18, 15:03:38
Last Released	2025-02-12, 12:57:37

Categories

Programming Languages

Snippets

Linters

Debuggers

Formatters

Testing

Resources

[Marketplace](#)

[Issues](#)

[Repository](#)

[License](#)

[Oracle Corporation](#)

Рисунок 0.2 – Вперше встановлене розширення

Але це розширення додавало кнопки “Run main” та “Debug main” до головного методу класу, хоч і працювало коректно. Тому було встановлено інше розширення: Language Support for Java від Red Hat.



Language Support for Java

Red Hat  redhat.com

 42,906,942

Java Linting, Intellisense, formatting, refactor...

Disable 

Uninstall 

Switch to Pre-Release Version

DETAILS

FEATURES

CHANGELOG

Language support for Java™ for Visual Studio Code

VS MARKETPLACE

INVALID

VS MARKETPLACE

RATE LIMITED BY UPSTREAM SERVICE

 CHAT

ON GITTER

 BUILD

PASSING

 LICENSE

EPL-2.0

Provides Java™ language support via Eclipse™ JDT Language Server, which utilizes Eclipse™ JDT, M2Eclipse and Buildship.

Quick Start

1. Install the Extension
2. On the following platforms, the extension should activate without any setup : **win32-x64**, **darwin-x64**, **darwin-arm64**, **linux-x64**, **linux-arm64**.

If on another platform, or using the "universal" version, you can set a



Installation

Identifier	redhat.java
Version	1.39.0
Last Updated	2025-02-26, 19:44:25
Size	166.18MB
Cache	16.58MB

Marketplace

Published	2016-09-12, 23:41:27
Last Released	2025-02-26, 10:30:01

Categories

Programming Languages

Linters

Formatters

Snippets

Resources

Marketplace

Issues

Repository

License

Red Hat

Рисунок 0.3 – Встановлене нове розширення

1 Вивід чисел паліндромів

Під час виконання були такі складнощі:

- У мові Java порівняння двох рядків за значенням має здійснюватися через метод `.equals()`. Після спроби їх порівняти оператором `==` програма не працювала;
- Для отримання даних від користувача потрібно створювати об'єкт типу `Scanner` і використовувати його методи;
- Для динамічного додавання даних до масиву потрібно використовувати інший тип даних: `List`; та ініціалізувати його типом `ArrayList` для використання його методу `.add()`;

Код

Спочатку код виглядав так:

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

class One {
    public static boolean isNumberPalindrome(int number) {
        String stringNumber = Integer.toString(number);
        String reversedNumber = "";

        for (int i = stringNumber.length() - 1; i >= 0; i--) {
            reversedNumber += stringNumber.charAt(i);
        }

        if (stringNumber.equals(reversedNumber))
            return true;
        else
            return false;
    }

    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.print("Please enter N, number of digits: ");
```

```

int numberOfDigits = inputScanner.nextInt();

int[] numbersArray = new int[numberOfDigits];
for (int i = 0; i < numbersArray.length; i++) {
    int readableNumber = i + 1;
    System.out.print("Please enter digit number " + readableNumber +
": ");
    int thisNumber = inputScanner.nextInt();
    numbersArray[i] = thisNumber;
}
inputScanner.close();

List<Integer> palindromeNumbers = new ArrayList<Integer>();
for (int i = 0; i < numbersArray.length; i++) {
    int thisNumber = numbersArray[i];
    boolean isThisNumberPalindrome = isNumberPalindrome(thisNumber);
    if (isThisNumberPalindrome)
        palindromeNumbers.add(thisNumber);
}

System.out.println("\nPalindrome numbers:");
for (int i = 0; i < palindromeNumbers.size(); i++) {
    System.out.println("- " + palindromeNumbers.get(i));
}
}
}

```

Після кількох змін його фінальний вигляд такий:

```

import java.util.Scanner;

class One {
    public static boolean isNumberPalindrome(int number) {
        String stringNumber = "" + number;
        String reversedString = new
StringBuilder(stringNumber).reverse().toString();

        if (stringNumber.equals(reversedString))
            return true;
        else
            return false;
    }

    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.print("Please enter N, number of digits: ");
        int numberOfDigits = inputScanner.nextInt();

        int[] numbersArray = new int[numberOfDigits];
        for (int i = 0; i < numbersArray.length; i++) {
            int readableNumber = i + 1;
            System.out.print("Please enter digit number " + readableNumber +
": ");
            int thisNumber = inputScanner.nextInt();

```



```

        numbersArray[i] = thisNumber;
    }
    inputScanner.close();

    System.out.println("\nPalindrome numbers:");
    for (int i = 0; i < numbersArray.length; i++) {
        int thisNumber = numbersArray[i];
        boolean isThisNumberPalindrome = isNumberPalindrome(thisNumber);
        if (isThisNumberPalindrome)
            System.out.println("- " + thisNumber);
    }
}
}

```

Результати

```

Please enter N, number of digits: 3
Please enter digit number 1: 333
Please enter digit number 2: 121
Please enter digit number 3: 513

Palindrome numbers:
- 333
- 121

```

Рисунок 1.1 – Вивід у консоль

2 Заповнення діагоналі матриці

Програма приймає від користувача квадратну матрицю заданого розміру, додає всі елементи як ключі структури даних TreeMap (яка їх сортує у спадному порядку), проходить через усі ключі і переставляє найбільші елементи на свої позиції у діагоналі.

Код

```

import java.util.Collections;
import java.util.Scanner;

```

```

import java.util.TreeMap;

class Two {
    public static void showMatrix(int[][] matrix) {
        System.out.println();
        for (int row = 0; row < matrix.length; row++) {
            for (int col = 0; col < matrix.length; col++) {
                if (row == col) {
                    System.out.print("\033[0;1m" + matrix[row][col] + "\033[0m
");
                } else {
                    System.out.print(matrix[row][col] + " ");
                }
            }
            System.out.println();
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Scanner inputReader = new Scanner(System.in);

        // Отримання розміру матриці
        int matrixSize;
        System.out.print("Please enter matrix size: ");
        matrixSize = inputReader.nextInt();

        // Заповнення
        int[][] matrix = new int[matrixSize][matrixSize];
        TreeMap<Integer, String> numbers = new TreeMap<Integer,
String>(Collections.reverseOrder());
        for (int row = 0; row < matrix.length; row++) {
            for (int col = 0; col < matrix.length; col++) {
                int readableRow = row + 1;
                int readableCol = col + 1;
                System.out.print("Please enter element on row " + readableRow +
" and col " + readableCol + ": ");

                int givenElement = inputReader.nextInt();
                matrix[row][col] = givenElement;
                String position = row + "," + col;
                numbers.put(givenElement, position);
            }
        }
        showMatrix(matrix);

        int matrixSizeCounter = 0;
        for (Integer key : numbers.keySet()) {
            if (matrixSizeCounter >= matrixSize) {
                break;
            }

            String value = numbers.get(key);
            String[] positionParts = value.split(",");
            int positionRow = Integer.parseInt(positionParts[0]);
            int positionCol = Integer.parseInt(positionParts[1]);
            System.out.println(key + ": row " + positionRow + " and col " +
positionCol);
        }
    }
}

```

```

        int neededRow = matrixSizeCounter;
        int neededCol = matrixSizeCounter;
        String neededPosition = neededRow + "," + neededCol;
        System.out.println("Needs to be on row " + neededRow + " and on
col " + neededCol);

        int swappedElement = matrix[neededRow][neededCol];
        int currentElement = matrix[positionRow][positionCol];
        matrix[neededRow][neededCol] = currentElement;
        matrix[positionRow][positionCol] = swappedElement;

        showMatrix(matrix);
        numbers.put(key, neededPosition);
        matrixSizeCounter += 1;
    }

    inputReader.close();
}

```

Результати

```
Please enter matrix size: 3
Please enter element on row 1 and col 1: 0
Please enter element on row 1 and col 2: 7
Please enter element on row 1 and col 3: 5
Please enter element on row 2 and col 1: 3
Please enter element on row 2 and col 2: 0
Please enter element on row 2 and col 3: 0
Please enter element on row 3 and col 1: 0
Please enter element on row 3 and col 2: 0
Please enter element on row 3 and col 3: 0
```

```
0 7 5
3 0 0
0 0 0
```

```
7: row 0 and col 1
Needs to be on row 0 and on col 0
```

```
7 0 5
3 0 0
0 0 0
```

```
5: row 0 and col 2
Needs to be on row 1 and on col 1
```

```
7 0 0
3 5 0
0 0 0
```

```
3: row 1 and col 0
Needs to be on row 2 and on col 2
```

```
7 0 0
0 5 0
0 0 3
```

Рисунок 2.1 – Вивід у консоль

3 Пошук локальних мінімумів

Завдання:

Знайти число локальних мінімумів. (Сусідами елемента матриці назвемо елементи, що мають із ним загальну сторону або кут. Елемент матриці називається локальним мінімумом, якщо він строго менше всіх своїх сусідів).

Код

```
import java.util.Scanner;
import java.util.TreeMap;

class Three {
    public static void showMatrix(int[][] matrix) {
        System.out.println();
        for (int row = 0; row < matrix.length; row++) {
            for (int col = 0; col < matrix.length; col++) {
                System.out.print(matrix[row][col] + " ");
            }
            System.out.println();
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Scanner inputReader = new Scanner(System.in);
        int matrixSize;
        System.out.print("Please enter matrix size: ");
        matrixSize = inputReader.nextInt();

        int[][] matrix = new int[matrixSize][matrixSize];
        for (int row = 0; row < matrix.length; row++) {
            for (int col = 0; col < matrix.length; col++) {
                int readableRow = row + 1;
                int readableCol = col + 1;
                System.out.print("Please enter element on row " + readableRow +
                    " and col " +
                        readableCol + ": ");

                int givenElement = inputReader.nextInt();
                matrix[row][col] = givenElement;
            }
        }
        showMatrix(matrix);
    }
}
```

```

TreeMap<String, Integer> mins = new TreeMap<String, Integer>();
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix.length; j++) {
        int thisElement = matrix[i][j];
        int minIndex = 0;
        int maxIndex = matrixSize - 1;

        int topLeft = Integer.MAX_VALUE;
        int top = Integer.MAX_VALUE;
        int topRight = Integer.MAX_VALUE;
        int right = Integer.MAX_VALUE;
        int bottomRight = Integer.MAX_VALUE;
        int bottom = Integer.MAX_VALUE;
        int bottomLeft = Integer.MAX_VALUE;
        int left = Integer.MAX_VALUE;

        if (i != minIndex && j != minIndex) {
            topLeft = matrix[i - 1][j - 1];
        }
        if (i != minIndex) {
            top = matrix[i - 1][j];
        }
        if (i != minIndex && j != maxIndex) {
            topRight = matrix[i - 1][j + 1];
        }
        if (j != maxIndex) {
            right = matrix[i][j + 1];
        }
        if (j != maxIndex && i != maxIndex) {
            bottomRight = matrix[i + 1][j + 1];
        }
        if (i != maxIndex) {
            bottom = matrix[i + 1][j];
        }
        if (i != maxIndex && j != minIndex) {
            bottomLeft = matrix[i + 1][j - 1];
        }
        if (j != minIndex) {
            left = matrix[i][j - 1];
        }

        if (thisElement < topLeft && thisElement < top && thisElement <
topRight && thisElement < right
            && thisElement < bottom && thisElement < bottomRight &&
thisElement < bottomLeft && thisElement < left) {
            String position = i + "," + j;
            mins.put(position, thisElement);
        }
    }
}

System.out.println("Local minimums:");
for (String key : mins.keySet()) {
    Integer value = mins.get(key);
    String[] valueParts = key.split(",");
    int row = Integer.parseInt(valueParts[0]) + 1;
    int col = Integer.parseInt(valueParts[1]) + 1;

```

```
        System.out.println("- Element " + value + " on row " + row + "
and col " + col);
    }

    inputReader.close();
}
}
```

Результати

```
Please enter matrix size: 3
Please enter element on row 1 and col 1: 0
Please enter element on row 1 and col 2: 1
Please enter element on row 1 and col 3: 1
Please enter element on row 2 and col 1: 1
Please enter element on row 2 and col 2: 1
Please enter element on row 2 and col 3: 1
Please enter element on row 3 and col 1: 1
Please enter element on row 3 and col 2: 1
Please enter element on row 3 and col 3: 0
```

```
0 1 1
1 1 1
1 1 0
```

```
Local minimums:
- Element 0 on row 1 and col 1
- Element 0 on row 3 and col 3
```

Рисунок 3.1 – Вивід у консоль

ВИСНОВКИ

Після виконання роботи, благодаттю Господа нашого Ісуса Христа, було ознайомлено з основами роботи з мовою Java та розробки програм на ній.

ПИТАННЯ

Типи даних в мові Java

Типи даних можна розділити на дві категорії: примітивні та посилальні.

Примітивні типи даних:

- int;
- boolean;
- char;
- byte;
- short;
- long;
- float;
- double.

Посилальні типи даних:

- String;
- Array;
- HashMap;
- ...

Типи даних колекцій не можуть приймати примітивні типи для їх оголошення.

Джерело: [Java Data Types at W3Schools.com](https://www.w3schools.com/java/java_data_types.asp).

Логічні операції в мові Java

Мова Java має 3 логічні операції:

- ТА: визначається оператором `&&`, повертає `true` якщо обидва вирази є правдою:


```
if (user.isLoggedIn && user.isRegistered)
    System.out.println("Welcome");
> Welcome
```

- АБО: визначається оператором `||`, повертає `true` якщо хоча б один вираз є правдою:

```
if (account.isDeleted || account.balance == 0)
    System.out.println("Cannot perform");
> Cannot perform
```

- НІ: оператор `!`, повертає `true` якщо вираз НЕ є правдою:

```
if (!car.isLoaded)
    System.out.println("Not loaded");
> Not loaded
```

Джерело: [Java Logical Operators in Java Operators at W3Schools.com](https://www.w3schools.com/java/java_logical_operators.asp)

Побітові операції в мові Java

У мові Java доступні такі побітові операції:

- Побітове АБО: оператор `|`, повертає 1 якщо один з бітів дорівнює 1;
- Побітове ТА: оператор `&`, повертає 1 якщо обидва біти є 1;
- Побітове виключне АБО: оператор `^`, повертає 1 якщо біти різні;
- Побітове доповнення: оператор `~`, повертає 0 якщо біт є 1 і 1 якщо біт є 0.

Приклад використання побітових операцій:

```
int a = 7;
int b = 3;
System.out.println("a: " + Integer.toBinaryString(a));
System.out.println("b: " + Integer.toBinaryString(b));
int and = a & b;
```

```
System.out.println(and + " AND in bin: " +
Integer.toBinaryString(and));
int or = a | b;
System.out.println(or + " OR in bin: " + Integer.toBinaryString(or));
int eor = a ^ b;
System.out.println(eor + " EOR in bin: " +
Integer.toBinaryString(eor));
int com = ~a;
System.out.println(com + " COM in bin: " +
Integer.toBinaryString(com));
> a: 111
> b: 11
> 3 AND in bin: 11
> 7 OR in bin: 111
> 4 EOR in bin: 100
> -8 COM in bin: 111111111111111111111111111111000
```

Джерело: [Bitwise Operators in Java – GeeksForGeeks](https://www.geeksforgeeks.org/bitwise-operators-in-java/).

Операції зсуву в мові Java

Мова Java дозволяє використовувати такі операції зсуву:

- Позначений зсув вліво: оператор `<<`, пересуває всі біти на зазначене число вліво: `variable << numberOfPlacesToShift;`
- Позначений зсув вправо: оператор `>>`, пересуває всі біти на зазначене число вправо: `variable >> numberOfPlacesToShift;`
- Непозначений зсув вправо: оператор `>>>`, пересуває всі біти на зазначене число вправо без урахування знаку: `variable >>> numberOfPlacesToShift.`

Приклад використання операцій зсуву:

```
int a = 7;
int b = 3;
System.out.println("a: " + Integer.toBinaryString(a));
System.out.println("b: " + Integer.toBinaryString(b));

int left = a << b;
int right = a >> b;
int unsignedRight = a >>> b;
System.out.println("Left << " + Integer.toBinaryString(left));
System.out.println("Right >> " + Integer.toBinaryString(right));
```

```
System.out.println("Unsigned right >>> " +  
Integer.toBinaryString(unsignedRight));  
> a: 111  
> b: 11  
> Left << 111000  
> Right >> 0  
> Unsigned right >>> 0
```

Джерело: [Shift Operators in Java – GeeksForGeeks.](#)

Правила написання імен в Java

Загальні правила для написанні імен:

- Має починатися з букви;
- Може містити літери, цифри, підкреслювання, ...;
- Краще писати, використовуючи `camelCase`;
- Не можна використовувати зарезервовані слова: `int`, `boolean`, ...;
- Враховують регістр.

Джерело: [Java Identifiers at W3Schools.com.](#)